

TO-DO LIST MANAGER - PROJECT REPORT

Submitted by: - Parminder Singh Bhatia

Registration no.: - 26BCE11648

Course: - Introduction to Problem Solving and Programming

Platform: - VITyarthi - Build Your Own Project

1. Introduction: -

The To-Do List Manager is a command-line application built in pure Python, using only the standard library. It was created to solve a small but common problem: forgetting the little day-to-day tasks that never get written down anywhere organized. The application lets a user quickly add tasks with a priority level, view them sorted so the most important ones show first, mark them complete, delete them, and check a summary of overall progress - all from a simple terminal menu, with no internet connection, database, or external software required. It is aimed primarily at students and anyone who wants a lightweight way to track daily to-do items without installing a phone app.

2. Problem Statement: -

The Problem:

Small daily tasks are easy to forget when they are not written down in an organized way. Phone apps and paper notes both add friction - one requires unlocking a device and finding the right app, the other is easy to misplace or lose. There was a need for a lightweight tool that could be opened instantly from a terminal, let tasks be added and reviewed in seconds, and still remember everything the next time the program was run.

The Solution:

A pure Python, command-line application that stores tasks with a title, priority, and completion status in a plain text file, sorts them automatically by priority when viewed, and provides a quick summary report of overall progress - all without requiring a database, external packages, or a graphical interface.

3. Functional Requirements: -

The system provides four major functional modules:

- Task Management (Add / View / Complete / Delete)
- Priority-Based Sorting (High / Medium / Low)

- Summary Reporting (totals, completed vs. pending, breakdown by priority)
- Data Persistence (plain text file storage)

4. Non-Functional Requirements: -

These define how the system performs its functions:

- **Usability:** A numbered menu (1-6) keeps every action a single keypress away, with no commands to memorize.
- **Reliability:** Invalid input - such as entering letters where a task ID is expected, or leaving the title blank - is handled gracefully with a clear message instead of an unhandled crash.
- **Portability:** The project uses only the Python standard library (os), so it runs anywhere Python 3 is installed with no extra packages to download.
- **Maintainability:** The code is split into focused modules - task_manager.py, file_handler.py, validator.py, and report.py - so each responsibility can be tested and updated independently.

5. System Architecture: -

The project follows a simple layered architecture, separating concerns into distinct parts:

- **Presentation Layer:** main.py - a terminal menu loop that handles user input and prints formatted output.
- **Logic Layer:** task_manager.py and validator.py - functions that add, sort, complete, and delete tasks, and validate user input.
- **Data Layer (Storage):** tasks.txt, managed by file_handler.py - a plain text file that stores all tasks persistently between runs.

6. Design Diagrams

6.1 Use Case Diagram: -

- **Actor:** User

Use Cases:

- Add a Task
- View Tasks (sorted by priority)
- Mark a Task as Complete
- Delete a Task
- View Summary Report
- **System:** Save & Load Tasks from File

6.2 Workflow Diagram

Start -> Launch App -> Load Tasks from tasks.txt -> User Selects Menu Option (Add / View / Complete / Delete / Summary) -> Validate Input -> Update Task List -> Save to File -> Display Result -> End.

6.3 Sequence Diagram

- User selects “Add Task” and enters a title and priority.
- Validator checks the title is not empty and defaults an unrecognized priority to Medium.
- Task Manager assigns a new task ID and appends the task to the in-memory list.
- File Handler writes the updated list to tasks.txt.
- Menu refreshes and confirms the task was added, ready for the next action.

7. Design Decisions & Rationale: -

- **Language:** Python was chosen for its simplicity and the fact that the entire project needed nothing beyond the standard library.
- **Command-Line Interface:** A CLI was chosen to keep the project focused on correct logic and reliable data handling, without the extra overhead of building a graphical interface.
- **Plain Text File over a Database:** tasks.txt was chosen instead of a database (or even JSON) to keep the project dependency-free, human-readable, and simple to inspect or edit by hand if needed.
- **Modular File Layout:** Splitting the project into task_manager.py, file_handler.py, validator.py, and report.py keeps each file focused on one job, which made writing unit tests much simpler.

8. Screenshots / Results: -

1. Adding a new task with a title and priority

```
===== TO-DO LIST MANAGER =====
1. Add Task
2. View Tasks
3. Mark Task as Complete
4. Delete Task
5. Exit
Choose an option (1-5): 1
Enter task description: study
Priority (High/Medium/Low) high
```

2. Viewing tasks sorted automatically by priority

```
===== TO-DO LIST MANAGER =====
1. Add Task
2. View Tasks
3. Mark Task as Complete
4. Delete Task
5. Exit
Choose an option (1-5): 2

--- Your Tasks ---
[1] (High) study - X Pending
```

3. Marking a task complete and viewing the updated summary report

```
===== TO-DO LIST MANAGER =====
1. Add Task
2. View Tasks
3. Mark Task as Complete
4. Delete Task
5. Exit
Choose an option (1-5): 3

--- Your Tasks ---
[1] (High) study - X Pending

Enter task ID to mark complete: 1
Task 1 marked as complete.
```

9. Testing Approach: -

- **Unit Testing:** Automated tests, run with unittest, cover the validation logic (empty titles, invalid priorities, invalid IDs) and core operations (adding, completing, and deleting tasks).
- **Mocking:** unittest.mock is used to prevent tests from writing to the real tasks.txt file, keeping test runs isolated and repeatable.
- **Manual Testing:** The menu was walked through end-to-end - adding tasks of each priority, viewing the sorted list, completing and deleting by ID, and checking the summary numbers update correctly.

10. Challenges Faced: -

- Deciding how to store structured task data (title, priority, completion status) in a plain text file without reaching for JSON or a database, while keeping it easy to parse back in.
- Writing unit tests that check file-handling behavior without actually touching the real tasks.txt file on disk - solved using unittest.mock.

- Handling every place a user could type something unexpected (blank titles, unrecognized priorities, non-numeric IDs) without the program ever crashing outright.

11. Learnings & Key Takeaways: -

- Learned how to structure a small Python project into focused, testable modules instead of one long script.
- Gained practical experience writing unit tests with unittest and unittest.mock to isolate file I/O from logic.
- Understood the value of input validation and sensible defaults (like defaulting an unrecognized priority to Medium) in keeping a program robust.

12. Future Enhancements: -

- **Due Dates & Reminders:** Add optional due dates and simple reminders for upcoming or overdue tasks.
- **Graphical Interface:** Build a Tkinter GUI on top of the existing task_manager.py logic, similar to other projects in this series.
- **Multiple Lists / Accounts:** Support separate task lists for different users or different areas of life (e.g. Work vs. Personal).

13. References

- **Python Documentation:** <https://docs.python.org/3/>
- **Python unittest Documentation:**
<https://docs.python.org/3/library/unittest.html>
- **Python unittest.mock Documentation:**
<https://docs.python.org/3/library/unittest.mock.html>